

Compliance Buddy

High-Level & Low-Level Design

System	Compliance Buddy (CB) — Automated Security Remediation
Version	1.0 — PR Review Loop Edition
Date	June 2026
Author	CB Engineering Team · skp4ml@gmail.com
Codebase	E:\CB_Automate
Runtime	k3d Kubernetes · Spring Boot 3.3.6 · Java 17
AI Models	GPT-4o (primary) · Claude Sonnet (fallback)
Repo	github.com/Skpandey15/kafka-micro-lab (target)

Table of Contents

PART I — HIGH-LEVEL DESIGN

1. System Overview & Goals
2. Architecture Overview
3. Service Catalogue
4. End-to-End Pipeline
5. Three-Reviewer PR Review Loop
6. Infrastructure & Deployment
7. Security Model
8. Resilience & Fault Tolerance

PART II — LOW-LEVEL DESIGN

9. Data Models
10. State Machines
11. cb-scanner
12. cb-agent
13. cb-patcher
14. cb-pr & AIReviewService
15. ReviewFeedbackPollerService
16. Prompt Engineering
17. cb-api REST Endpoints
18. cb-notifier
19. Kubernetes Manifests
20. Key Patterns & Gotchas

PART I

High-Level Design

System goals, architectural decisions, component overview, and the end-to-end pipeline

1. System Overview & Goals

1.1 Purpose

Compliance Buddy (CB) is an **autonomous security remediation pipeline** that:

1. Continuously scans Java codebases via SonarQube for security vulnerabilities (CWEs, OWASP Top 10).
2. Uses GPT-4o to generate a code patch (unified diff) that remediates each vulnerability.
3. Applies the patch to a local clone of the repository, runs build validation, and pushes a PR to GitHub.
4. Conducts a **3-reviewer AI+human code review loop**: GitHub Copilot, a configured human reviewer, and CB's own GPT-4o self-review with ACCEPT/REJECT checkboxes.
5. On rejection, incorporates reviewer feedback into a re-generated fix — closing the remediation loop automatically.

✓ End-to-end tested and live

The full pipeline from SonarQube scan → GPT-4o fix → validated patch → PR creation → CB review comment → ACCEPT/REJECT polling is operational on the kafka-micro-lab target repository.

1.2 Design Goals

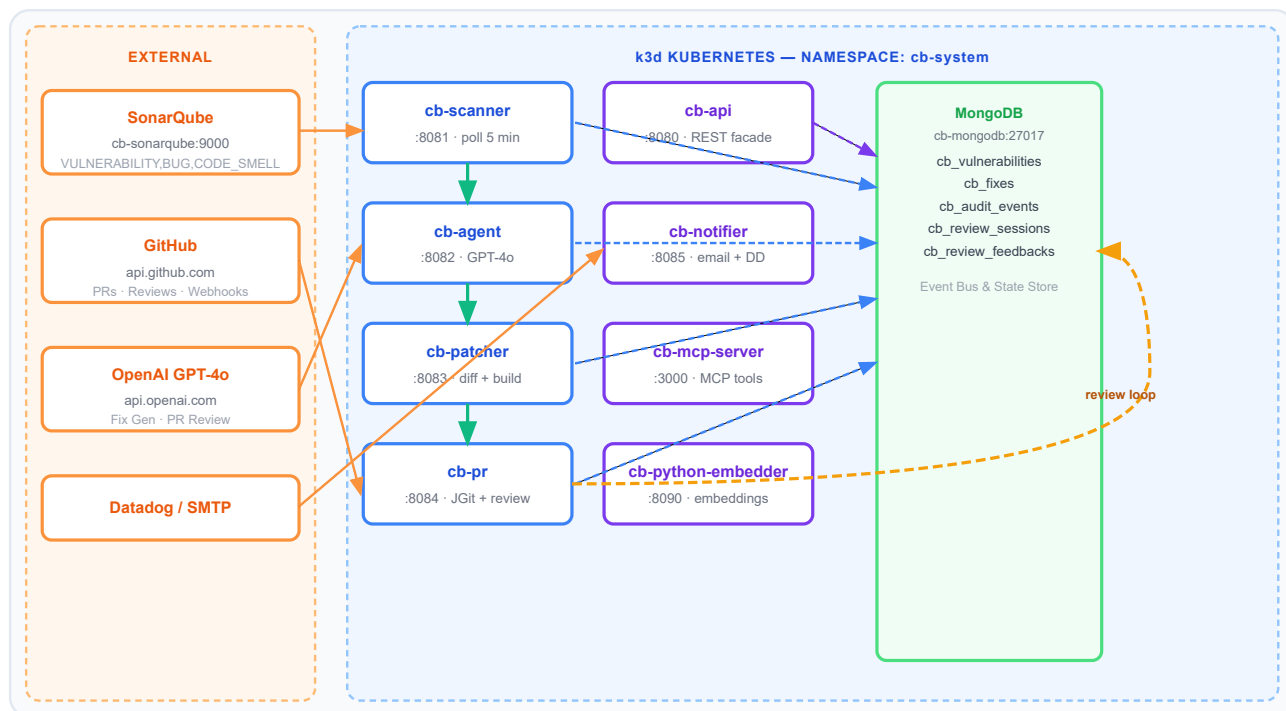
Goal	Design Decision
Zero human toil for routine CVEs	Fully automated: scanner → LLM → patch → PR with no human in the loop until review time
No blocking in schedulers	GitHub API + GPT-4o calls run in <code>CompletableFuture.runAsync()</code> — scheduler thread returns immediately
Cross-pod communication	MongoDB as the inter-service message bus; each service polls its own status slice (Spring events are in-process only)
Reviewer feedback loop	Rejection reason stored in <code>cb_review_feedbacks</code> ; cb-agent reads it before re-generating the fix
Resilience	Resilience4j circuit breaker + retry on all LLM and GitHub API calls; 10s connect / 30s read timeouts on RestClient
Local dev & CI parity	Entire stack (SonarQube, MongoDB, all 8 services) runs in k3d on a developer laptop

1.3 Non-Goals (v1)

- Multi-language support (Java only in v1)
- Auto-merge without human ACCEPT (human must tick the checkbox)
- SonarQube auto-scanner triggering (scans are scheduled; on-demand via REST API)

2. Architecture Overview

2.1 High-Level Architecture



2.2 Key Architectural Principles

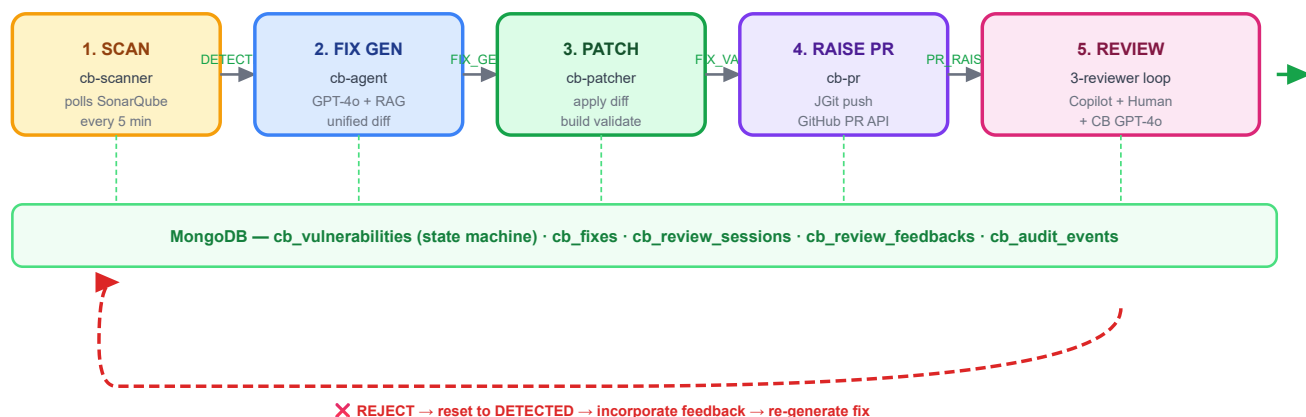
Principle	Implementation	Rationale
MongoDB as message bus	Each service polls a status slice of <code>cb_vulnerabilities</code> on a schedule	Spring <code>ApplicationEvent</code> is in-process; events don't cross pod boundaries in k8s
Fire-and-forget async	<code>CompletableFuture.runAsync()</code> for GPT-4o review & GitHub reviewer calls after PR creation	Prevents the <code>@Scheduled</code> <code>synchronized</code> PR poller thread from blocking on network I/O
Immutable domain records	All domain objects are Java <code>record</code> s; state transitions return new instances via <code>withStatus()</code>	Thread-safety, easy audit trail, no partial-update bugs
Retry + circuit breaker	Resilience4j annotations on all LLM and GitHub API calls	Temporary API outages don't cascade to failed remediation
No direct inter-service HTTP	Services only call external APIs (SonarQube, GitHub, OpenAI); no service-to-service REST	Avoids distributed tracing complexity; simpler failure modes

3. Service Catalogue

Service	Port	Role	Key External Dependency
cb-core	—	Shared library: domain records, repositories, events, async config, audit service	MongoDB
cb-api	8080	REST facade — exposes scan triggers, vulnerability/fix CRUD, audit log, metrics	MongoDB, ScannerService bean
cb-scanner	8081	Polls SonarQube every 5 min for new issues; deduplicates; saves DETECTED vulnerabilities	SonarQube :9000
cb-agent	8082	Polls DETECTED vulns; calls GPT-4o (with RAG context + optional rejection feedback) to generate unified-diff fixes	OpenAI GPT-4o, cb-python-embedder
cb-patcher	8083	Polls FIX_GENERATED; applies diff to /workspace/repo; runs build validation; marks FIX_VALIDATED	/workspace/repo (git clone)
cb-pr	8084	Polls FIX_VALIDATED; commits+pushes fix branch via JGit; creates GitHub PR; posts CB AI review comment; polls for ACCEPT/REJECT	GitHub API, OpenAI GPT-4o
cb-notifier	8085	Listens to PRRaisedEvent & EscalationEvent (in-process); sends email + Datadog event	SMTP, Datadog API
cb-mcp-server	3000	MCP tool server exposing CB capabilities as AI tools (code context, vulnerability lookup)	MongoDB
cb-python-embedder	8090	REST service wrapping sentence-transformers for generating vulnerability embedding vectors for RAG	HuggingFace model (local)

4. End-to-End Pipeline

4.1 Main Remediation Flow



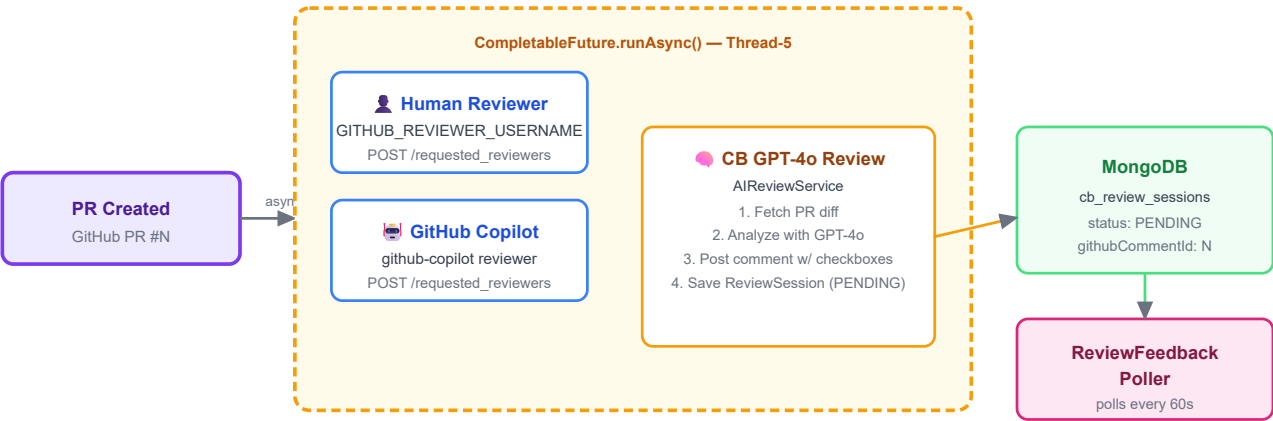
4.2 Status Transition Summary

From	To	Trigger Service	Condition
—	DETECTED	cb-scanner	New SonarQube issue, not already in DB
DETECTED	IN_PROGRESS	cb-agent	VulnerabilityDetectedEvent fires on poll
IN_PROGRESS	FIX_GENERATED	cb-agent	GPT-4o returns valid JSON-LD fix
IN_PROGRESS	FAILED	cb-agent	LLM failure after max retries
FIX_GENERATED	FIX_VALIDATED	cb-patcher	Diff applies cleanly and build passes
FIX_GENERATED	FAILED	cb-patcher	Diff conflict or build failure
FIX_VALIDATED	PR_RAISED	cb-pr	PR created on GitHub
PR_RAISED	DETECTED	cb-pr (reject loop)	Human checks ✗ REJECT checkbox on PR
PR_RAISED	RESOLVED	Manual / webhook	PR merged

5. Three-Reviewer PR Review Loop

5.1 Overview

After cb-pr creates a pull request, it fires three review actions **asynchronously** on a background thread so the scheduler is not blocked:



5.2 ACCEPT / REJECT Decision Flow

The `ReviewFeedbackPollerService` polls all PENDING `ReviewSession` documents every 60 seconds and reads the GitHub comment body via `GET /repos/:owner/:repo/issues/comments/:id`.

GitHub updates the comment body in-place when a user clicks a checkbox: `- [ ]` becomes `- [x]`.

Decision	How to Signal	CB Action
ACCEPT	Check <code>[]</code> ☒ <code>ACCEPT</code> on the PR comment	<code>ReviewSession</code> → <code>ACCEPTED</code> . PR proceeds to normal merge.
REJECT	Check <code>[]</code> ☐ <code>REJECT</code> + post comment: <code>REJECT: <reason></code>	- Save <code>ReviewFeedback</code> (rejectionReason, processed=false) - Close PR via <code>PATCH /pulls/:id {state: closed}</code> - Reset vuln to <code>DETECTED</code> (retryCount++) - cb-agent re-picks up → reads rejection feedback → augments prompt → new fix → new PR

6. Infrastructure & Deployment

6.1 Kubernetes Topology

Resource	Spec	Notes
Cluster	k3d <code>compliance-buddy</code>	1 server node + 2 agent nodes; binary at <code>C:\Users\skp4j\bin\k3d</code>
Namespace	<code>cb-system</code>	All services and MongoDB in this namespace
Ingress	k3d built-in, host <code>compliance-buddy.local</code>	<code>/api</code> → cb-api; <code>/sonar</code> → SonarQube
MongoDB	Standalone, 1 replica	Data persists in <code>emptyDir</code> (dev only); collections created lazily on first insert
SonarQube	Community Edition, 1 replica	Pre-loaded with kafka-micro-lab project; admin/SonarQube!08
Images	Local Docker built, imported via k3d	<code>eclipse-temurin:17-jdk-jammy</code> builder → <code>17-jre-jammy</code> runtime
Security context	<code>runAsUser: 1001, runAsNonRoot: true, allowPrivilegeEscalation: false</code>	Applied to all CB service pods

6.2 ConfigMap / Secrets Split

- **cb-common-config** (ConfigMap): non-secret config — MongoDB URI, embedder URL, poll intervals, model names
- **cb-secrets** (Secret): API keys, tokens — OpenAI, Anthropic, GitHub, SonarQube, SMTP, Datadog

6.3 Deployment Procedure

```
# 1. Build
docker build --no-cache -f E:/CB_Automate/<module>/Dockerfile -t <module>:latest E:/CB_Automate

# 2. Import into k3d
C:\Users\skp4j\bin\k3d image import <module>:latest -c compliance-buddy

# 3. Rolling restart
kubectl rollout restart deployment/<module> -n cb-system
kubectl rollout status deployment/<module> -n cb-system --timeout=180s

# 4. Re-clone repo in cb-patcher and cb-pr after restart
kubectl exec -n cb-system <pod> -- sh -c \
    "git clone https://token:$TOKEN@github.com/Skpandey15/kafka-micro-lab /workspace/repo"
```

7. Security Model

Concern	Mitigation
API authentication	All REST endpoints protected by <code>X-API-Key</code> header filter (<code>ApiKeyAuthFilter</code>); key in Kubernetes secret
GitHub token scope	PAT with <code>repo</code> scope only; stored in <code>cb-secrets</code> , injected as env var
LLM prompt injection	System prompt enforces strict JSON-LD output format; parser rejects anything not matching schema
Container hardening	Non-root user (uid 1001), no privilege escalation, read-only app JAR
Secrets management	Kubernetes secrets (base64); Vault integration prepared but disabled (<code>VAULT_ENABLED=false</code>)
Git push credentials	Token injected at runtime via JGit <code>UsernamePasswordCredentialsProvider</code> ; never persisted in git config

8. Resilience & Fault Tolerance

Pattern	Where Applied	Config
Circuit Breaker	GitHub API calls (<code>createBranch</code> , <code>createPullRequest</code>), LLM calls in cb-agent	Resilience4j <code>@CircuitBreaker(name="github-api")</code> and <code>@CircuitBreaker(name="llm-agent")</code>
Retry	Same as above; LLM retry on transient errors	<code>@Retry(name="github-api")</code> , <code>@Retry(name="llm-agent")</code>
LLM Fallback	cb-agent: if GPT-4o circuit open	<code>callFallbackLlm()</code> uses Claude Sonnet 4.6
HTTP Timeout	All GitHub API calls from cb-pr	<code>SimpleClientHttpRequestFactory</code> : 10s connect, 30s read
Retry count limit	cb-agent fix generation	<code>MAX_RETRIES = 3</code> ; escalates to FAILED + EscalationEvent
RAG fallback	cb-agent vector search	MongoDB Atlas <code>\$vectorSearch</code> not available locally → empty RAG context, falls back to no-RAG prompt
Async error isolation	Reviewer request + AI review after PR creation	<code>CompletableFuture.exceptionally()</code> logs error; PR creation is NOT rolled back

PART II

Low-Level Design

Data models, state machines, per-service internals, prompt design, API contracts

9. Data Models

9.1 Vulnerability

Collection: `cb_vulnerabilities`

Field	Type	Description
<code>id</code>	String (ObjectId)	MongoDB primary key
<code>sonarIssueKey</code>	String (unique index)	SonarQube issue key — dedup guard
<code>cweId</code>	String	e.g. <code>CWE-89</code> , <code>CWE-1041</code>
<code>severity</code>	Severity enum	BLOCKER / CRITICAL / MAJOR / MINOR / INFO
<code>status</code>	VulnerabilityStatus enum	State machine field (see §10)
<code>filePath</code>	String	Relative path in target repo
<code>lineNo</code>	int	Line number of the vulnerability
<code>message</code>	String	SonarQube issue message
<code>owaspCategory</code>	String	OWASP mapping from ontology
<code>retryCount</code>	int	Fix generation attempts; incremented on REJECT or failure
<code>embeddingVector</code>	List<Double>	384-dim sentence-transformer embedding for RAG similarity search
<code>detectedAt</code>	Instant	First detection timestamp (<code>@CreatedDate</code>)
<code>updatedAt</code>	Instant	Last status change (<code>@LastModifiedDate</code>)
<code>resolvedAt</code>	Instant	Set when status → RESOLVED

9.2 Fix

Collection: `cb_fixes`

Field	Type	Description
<code>id</code>	String (ObjectId)	MongoDB primary key; first 12 chars used as branch name suffix
<code>vulnerabilityId</code>	String (indexed)	FK to <code>cb_vulnerabilities</code>
<code>patchDiff</code>	String	Unified diff generated by GPT-4o
<code>gradlePatch</code>	String (nullable)	build.gradle changes (dependency additions)
<code>strategy</code>	String	Short label e.g. <code>prepared-statement</code>
<code>confidence</code>	double	0.0–1.0 LLM self-confidence score
<code>llmModel</code>	String	e.g. <code>gpt-4o</code>
<code>explanation</code>	String	≤200 word explanation of the fix
<code>prUrl</code>	String (nullable)	GitHub PR HTML URL after PR creation
<code>prNumber</code>	Integer (nullable)	GitHub PR number
<code>branchName</code>	String (nullable)	e.g. <code>cb/fix-cwe-89-a3b4c5d6e7f8</code>
<code>status</code>	FixStatus enum	PENDING → BUILD_VALIDATED / BUILD_FAILED → PR_OPEN
<code>buildValidated</code>	boolean	True if build passed
<code>tokensUsed</code>	int	LLM tokens consumed
<code>generatedAt</code>	Instant	Fix generation timestamp

9.3 ReviewSession

Collection: `cb_review_sessions` — tracks the CB AI review comment on each PR

Field	Type	Description
<code>id</code>	String	MongoDB primary key
<code>vulnerabilityId</code>	String (indexed)	FK to <code>cb_vulnerabilities</code>
<code>prNumber</code>	int	GitHub PR number
<code>prUrl</code>	String	GitHub PR HTML URL
<code>githubCommentId</code>	Long	ID of the CB review comment — used to poll checkbox state
<code>status</code>	String	<code>PENDING</code> <code>NO_FINDINGS</code> <code>ACCEPTED</code> <code>REJECTED</code>
<code>cbFindingsCount</code>	int	Number of findings from GPT-4o review
<code>createdAt</code>	Instant	When the review was posted

9.4 ReviewFeedback

Collection: `cb_review_feedbacks` — persists rejection reasons for cb-agent re-fix

Field	Type	Description
<code>id</code>	String	MongoDB primary key
<code>vulnerabilityId</code>	String (indexed)	FK to <code>cb_vulnerabilities</code>
<code>prNumber</code>	int	The PR that was rejected
<code>rejectionReason</code>	String	Text from <code>REJECT: <reason></code> PR comment
<code>processed</code>	boolean	False until cb-agent reads it for re-fix; set true after
<code>createdAt</code>	Instant	Rejection timestamp

9.5 AuditEvent

Collection: `cb_audit_events` — immutable audit trail for every state change

Field	Description
<code>entityId, entityType</code>	What was changed (vulnId/fixId + "Vulnerability"/"Fix")
<code>action</code>	e.g. DETECTED, FIX_GENERATED, PR_RAISED, ESCALATED
<code>actor</code>	Service name that made the change
<code>metadata</code>	Arbitrary Map<String,Object> (e.g. model, confidence, prUrl)
<code>timestamp</code>	@CreatedDate

11. cb-scanner — Low-Level Design

11.1 Responsibilities

- Poll SonarQube API every 5 minutes (configurable via `SCANNER_POLL_INTERVAL_MS`)
- Page through all `VULNERABILITY`, `BUG`, `CODE_SMELL` issues for each configured project
- Deduplicate via `sonarIssueKey` unique index; save new ones as DETECTED
- Publish `VulnerabilityDetectedEvent` (in-process) after saving
- Accept on-demand scan from cb-api (`POST /api/v1/scans/{projectKey}`)

11.2 ScannerService — Key Method

```
@Scheduled(fixedDelayString = "${scanner.poll-interval-ms:300000}")
public void scheduledScan() {
    monitoredProjects.forEach(this::scanProject);
}

public int scanProject(String projectKey) {
    int page = 1, newFindings = 0;
    do {
        SonarIssuesResponse resp = sonarClient.getSecurityIssues(projectKey, page);
        for (SonarIssue issue : resp.issues()) {
            if (!vulnerabilityRepo.existsBySonarIssueKey(issue.key())) {
                Vulnerability vuln = issueMapper.toVulnerability(issue, projectKey);
                vulnerabilityRepo.save(vuln);
                eventPublisher.publishEvent(new VulnerabilityDetectedEvent(this, vuln));
                newFindings++;
            }
        }
        page++;
    } while ((page - 1) * 100L < resp.paging().total());
    return newFindings;
}
```

11.3 SonarQube Client

REST client hitting `http://cb-sonarqube:9000/sonar/api/issues/search` with query params:

`types=VULNERABILITY,BUG,CODE_SMELL&languages=java&componentKeys={project}&p={page}&ps=100`

12. cb-agent — Low-Level Design

12.1 Responsibilities

- Poll DETECTED vulnerabilities every 30s (via `VulnerabilityPollerService`)
- For each: fetch similar past fixes via RAG (sentence-transformer embeddings → MongoDB vector search)
- Fetch any unprocessed `ReviewFeedback` for re-fix scenarios
- Call GPT-4o with system + user prompt; parse JSON-LD `cb:Fix` response
- Mark feedbacks as processed; update status to `FIX_GENERATED`; fire `FixGeneratedEvent`

12.2 Fix Generation Flow

```
@Async("cbAgentExecutor") @EventListener
public void onVulnerabilityDetected(VulnerabilityDetectedEvent event) {
    vulnerabilityRepo.save(vuln.withStatus(IN_PROGRESS));

    List<Fix> ragFixes = ragService.retrieveSimilarFixes(vuln);
    List<ReviewFeedback> feedbacks =
        reviewFeedbackRepo.findByVulnerabilityIdAndProcessedFalse(vuln.id());

    String rawFix = callLlmWithFallback(vuln, ragFixes, feedbacks);
    Fix fix = parseFix(rawFix, vuln);
    fixRepo.save(fix);

    feedbacks.forEach(f -> reviewFeedbackRepo.save(f.withProcessed()));
    vulnerabilityRepo.save(vuln.withStatus(FIX_GENERATED));
    eventPublisher.publishEvent(new FixGeneratedEvent(this, vuln, fix));
}
```

12.3 RAG Retrieval Service

- Posts vulnerability text to `cb-python-embedder:8090/embed` to get 384-dim vector
- Queries `cb_fixes` with `$vectorSearch` (MongoDB Atlas) for top-3 similar BUILD_VALIDATED fixes
- Falls back gracefully to empty list on local MongoDB (Atlas-only operator)

12.4 LLM Configuration

Setting	Value
Primary model	<code>gpt-4o</code> (via Spring AI OpenAI starter)
Fallback model	<code>claude-sonnet-4-6</code> (via Spring AI Anthropic starter)
Circuit breaker name	<code>llm-agent</code>
Bean qualifier	<code>@Qualifier("primaryChatClient") / "fallbackChatClient"</code>
Prompt mode	Two-call design: system prompt (CWE guidelines) + user prompt (vuln JSON-LD + RAG examples)

13. cb-patcher — Low-Level Design

13.1 Responsibilities

- Listen to `FixGeneratedEvent` (in-process) — fired by cb-agent on the same pod
- Apply the unified diff to `/workspace/repo` using a custom parser
- Optionally apply `gradlePatch` to add new dependencies in `build.gradle`
- Run `BuildValidator.runBuild()` — executes `./gradlew compileJava --no-daemon` with 10-minute timeout
- Mark fix as BUILD_VALIDATED or BUILD_FAILED; update vulnerability status

⚠ Note on cross-pod events

PatcherService listens to `FixGeneratedEvent` via `@EventListener`. In the current k8s deployment, cb-agent and cb-patcher are

separate pods

. The in-process event only fires within cb-agent. cb-patcher uses its own `FixPollerService` which polls for `FIX_GENERATED` status in MongoDB every 30 seconds as the reliable cross-pod trigger.

13.2 DiffApplier Algorithm

1. Parse unified diff: extract file paths from `+++ b/path` headers and hunk headers (`@@ -L,N +L,N @@`)
2. For each hunk, compute target line index with an offset accumulator (accounting for previous hunk's line count delta)
3. Replace context+deleted lines with added lines via `List.subList().clear()` + `List.addAll()`
4. Write modified line list back to file

13.3 Build Validation

```
ProcessBuilder pb = new ProcessBuilder("./gradlew", "compileJava", "--no-daemon");
pb.directory(new File(repoRoot));
pb.redirectErrorStream(true);
Process process = pb.start();
boolean completed = process.waitFor(timeoutMinutes, TimeUnit.MINUTES);
boolean success = completed && process.exitValue() == 0;
```

Build log is captured (up to 10 000 chars) and stored in `Fix.buildLog`.

14. cb-pr & AIReviewService — Low-Level Design

14.1 PRPollerService

- Polls `FIX_VALIDATED` vulnerabilities every 60s
- `synchronized` — ensures only one PR is created at a time (shared `/workspace/repo`)
- For each `FIX_VALIDATED`: reset repo → apply diff → commit → push via JGit → create GitHub PR
- After PR creation: fires async task for reviewer requests + AI review

14.2 JGit Push Flow

```
// 1. Reset to main/HEAD
git.checkout().setName("main").call();
git.reset().setMode(HARD).setRef("HEAD").call();
git.clean().setCleanDirectories(true).setForce(true).call();

// 2. Apply diff
applyDiff(repoRoot, fix.patchDiff());

// 3. Branch, commit, push
git.checkout().setCreateBranch(true).setName(branchName).call();
git.add().addFilepattern(".").call();
git.commit().setMessage("fix(" + vuln.cweId() + "): Auto-remediated by Compliance Buddy")
    .setAuthor("Compliance Buddy", "cb-bot@techseva.in").call();
git.push().setCredentialsProvider(new UsernamePasswordCredentialsProvider("token", githubToken))
    .setRemote("origin").call();
```

14.3 Async Reviewer + Review Dispatch

```
// After PR creation – fire-and-forget so @Scheduled thread returns immediately
CompletableFuture.runAsync(() -> {
    requestReviewers(prNumber); // Copilot + optional human
    aiReviewService.reviewAndPost(vuln, fix, prNumber, prUrl); // GPT-4o + comment
}).exceptionally(ex -> {
    log.error("Async reviewer/review failed for PR #{}: {}", prNumber, ex.getMessage(), ex);
    return null;
});
```

14.4 AIReviewService — GPT-4o Review

1. Fetch PR diff via `GET /repos/:owner/:repo/pulls/:number` with `Accept: application/vnd.github.diff`
2. Call GPT-4o with system prompt (senior Java security engineer role) + diff content
3. Parse JSON array of findings: `[{"title", "severity", "description"}, ... ]`
4. Build GitHub comment body with findings + ACCEPT/REJECT markdown checkboxes
5. POST comment via `POST /repos/:owner/:repo/issues/:prNumber/comments`
6. Save `ReviewSession` to MongoDB with returned comment ID and status PENDING (or NO_FINDINGS)

14.5 GitHubEnterpriseClient Methods

Method	HTTP	Purpose
<code>createBranch</code>	GET + POST <code>/git/refs</code>	Create branch from HEAD SHA
<code>createPullRequest</code>	POST <code>/pulls</code>	Open PR with title/body/head/base
<code>requestReviewers</code>	POST <code>/pulls/:n/requested_reviewers</code>	Add Copilot + human reviewer
<code>getPRDiff</code>	GET <code>/pulls/:n</code> (diff Accept)	Fetch unified diff for AI review
<code>createIssueComment</code>	POST <code>/issues/:n/comments</code>	Post CB review comment
<code>getIssueCommentBody</code>	GET <code>/issues/comments/:id</code>	Read comment to check checkbox state
<code>getPRIssueComments</code>	GET <code>/issues/:n/comments</code>	Find <code>REJECT: <reason></code> comment
<code>closePR</code>	PATCH <code>/pulls/:n</code> state=closed	Close rejected PR

All methods use a shared `RestClient` with `SimpleClientHttpRequestFactory` (10s connect / 30s read timeout) and Bearer token auth.

15. ReviewFeedbackPollerService — Low-Level Design

15.1 Poll Loop

```
@Scheduled(fixedDelayString = "${review.poll-interval-ms:60000}", initialDelay = 30000)
public synchronized void pollPendingReviews() {
    List<ReviewSession> pending = reviewSessionRepo.findByStatus("PENDING");
    for (ReviewSession session : pending) {
        String body = githubClient.getIssueCommentBody(owner, repo, session.githubCommentId());
        if (isAccepted(body)) reviewSessionRepo.save(session.withStatus("ACCEPTED"));
        else if (isRejected(body)) handleRejection(session, extractRejectionReason(prNumber));
    }
}
```

15.2 Checkbox Detection

GitHub renders `- [ ]` as a clickable checkbox. When clicked, GitHub edits the comment body in-place, replacing `[ ]` with `[x]`.

Signal	String to Match
ACCEPT	<code>[x] ✓ **ACCEPT**</code> or <code>[X] ✓ ACCEPT</code>
REJECT	<code>[x] ✗ **REJECT**</code> or <code>[X] ✗ REJECT</code>
Rejection reason	Any PR comment starting with <code>REJECT:</code> (case-insensitive)

15.3 Rejection Handling

1. Query all PR comments looking for one starting with `REJECT:`
2. Save `ReviewFeedback{rejectionReason, processed=false}`
3. Close PR: `githubClient.closePR(owner, repo, prNumber)`
4. Reset vulnerability: `vuln.incrementRetryCount().withStatus(DETECTED)`
5. Save `ReviewSession.withStatus("REJECTED")`
6. cb-agent polls within 30s, reads the unprocessed `ReviewFeedback`, augments GPT-4o prompt

16. Prompt Engineering

16.1 System Prompt (Fix Generation)

The system prompt establishes the AI persona and strict output rules:

- Role: expert Java 17/Spring Boot security engineer, OWASP Top 10 specialist
- CWE-specific remediation guidelines for 8 common CWEs (SQL injection, XSS, path traversal, hardcoded credentials, etc.)
- Output format: JSON-LD `cb:Fix` object — exactly 6 required fields, no prose outside JSON
- Each patched line must include trailing comment `// CB-FIX: <cweId> - <strategy>`

16.2 User Prompt Structure

```
## VULNERABILITY TO REMEDIATE
{vulnerability as JSON-LD cb:Vulnerability}

## SIMILAR VALIDATED FIXES (RAG top-3) ← optional, if available
Adapt the fix style from these validated examples:
[{cb:Fix JSON-LD}, ...]

## ⚠️ PREVIOUS FIX REJECTED – REVIEWER FEEDBACK ← only on re-fix
Your previous fix attempt was rejected. You MUST address ALL of the following:
1. {rejectionReason1}
2. {rejectionReason2}
Generate an improved fix that explicitly resolves every concern listed above.

## INSTRUCTION
Generate a JSON-LD cb:Fix object. Follow CWE guidelines. Respond ONLY with JSON-LD.
```

16.3 AI Review System Prompt (cb-pr)

- Role: senior Java security engineer reviewing an auto-generated patch
- Goal: identify remaining concerns, missed cases, side effects
- Output: JSON array of findings (title ≤8 words, severity HIGH|MEDIUM|LOW, description ≤100 words)
- Clean fix → return `[]` (empty array)

17. cb-api REST Endpoints

Base URL: `http://compliance-buddy.local/api` | Auth: `X-API-Key: dev-key-change-in-prod`

Method	Endpoint	Description
POST	<code>/v1/scans/{projectKey}</code>	Trigger on-demand SonarQube scan
GET	<code>/v1/vulnerabilities</code>	List all vulnerabilities (with optional status filter)
GET	<code>/v1/vulnerabilities/{id}</code>	Get vulnerability by ID
GET	<code>/v1/fixes</code>	List all fixes
GET	<code>/v1/fixes/{id}</code>	Get fix by ID
GET	<code>/v1/audit</code>	Audit event log (sortable by timestamp)
GET	<code>/v1/metrics</code>	Pipeline metrics (counts by status, avg confidence, etc.)
GET	<code>/actuator/health</code>	Spring Boot health check
GET	<code>/swagger-ui</code>	OpenAPI / Swagger UI

18. cb-notifier — Low-Level Design

cb-notifier listens to in-process Spring events and sends notifications asynchronously on the `cbNotifierExecutor` thread pool.

Event	Notification	Condition
<code>PRRaisedEvent</code>	Email + Datadog event: "PR #N raised for CWE-X in file.java"	Every successful PR creation
<code>EscalationEvent</code>	Email + Datadog alert: "Fix generation failed after N retries"	MAX_RETRIES exceeded or build failure

Known limitation

Since cb-notifier is a separate pod from cb-pr (which fires `PRRaisedEvent`), in-process Spring events don't cross the pod boundary. A `NotifierPollerService` polling `cb_audit_events` for PR_RAISED entries is the correct long-term fix.

19. Kubernetes Manifests

19.1 Resource Summary

Service	CPU Request	CPU Limit	Memory Req	Memory Limit	Replicas
cb-scanner	100m	300m	256Mi	512Mi	1
cb-agent	200m	500m	512Mi	1Gi	1
cb-patcher	200m	500m	512Mi	1Gi	1
cb-pr	200m	400m	256Mi	512Mi	1
cb-api	100m	300m	256Mi	512Mi	1
cb-notifier	100m	200m	128Mi	256Mi	1
cb-mongodb	200m	500m	512Mi	1Gi	1
cb-sonarqube	500m	1000m	1Gi	2Gi	1

19.2 Volume Strategy

- cb-patcher & cb-pr: `emptyDir` volume mounted at `/workspace/repo` — must be re-cloned after pod restart
- MongoDB: `emptyDir` (dev environment) — data lost on pod restart; PVC recommended for production
- SonarQube: `emptyDir` for dev; H2 database embedded

19.3 Health Probes

```
livenessProbe:
  httpGet: { path: /actuator/health/liveness, port: 808X }
  initialDelaySeconds: 45
  periodSeconds: 20

readinessProbe:
  httpGet: { path: /actuator/health/readiness, port: 808X }
  initialDelaySeconds: 30
  periodSeconds: 10
```

20. Key Patterns & Gotchas

20.1 The Poller Pattern (Critical)

Why pollers instead of Spring events across services?

Spring's `ApplicationEventPublisher` / `@EventListener` is in-process only. In Kubernetes, each service runs in its own pod. Events published in cb-agent are invisible to cb-patcher or cb-pr.

Every cross-pod trigger must go through MongoDB status polling.

Service	Poller	Polls For	Interval
cb-scanner	ScannerService.scheduledScan()	SonarQube new issues	5 min
cb-agent	VulnerabilityPollerService	status = DETECTED	30s
cb-patcher	FixPollerService	status = FIX_GENERATED	30s
cb-pr	PRPollerService	status = FIX_VALIDATED	60s
cb-pr	ReviewFeedbackPollerService	ReviewSession status = PENDING	60s

20.2 Branch Naming Convention

```
branchName = "cb/fix-" + cweId.toLowerCase().replace(":", "-")
              + "-" + fixId.substring(0, 12)
// Example: cb/fix-cwe-89-a3b4c5d6e7f8
```

The 12-character fix ID suffix prevents branch name collisions between multiple fixes for the same CWE.

20.3 Known Limitations & Workarounds

Issue	Cause	Workaround
repo lost on pod restart	<code>emptyDir</code> volume cleared on restart	Re-clone after every cb-patcher/cb-pr restart
Vector search fails locally	<code>\$vectorSearch</code> is Atlas-only	Falls back gracefully to no-RAG prompt
Java 26 Gradle incompatibility	Host JDK 26 incompatible with Gradle 8.8	Always build inside Docker (eclipse-temurin:17)
JGit config write error	<code>/home/cbapp/.config/jgit</code> not writable	Non-fatal; <code>setAuthor()</code> still works; safe to ignore
Multiple TaskExecutor warning	cb-core AsyncConfig defines 4 named executors	Add <code>@Primary</code> to the default executor if needed; harmless in current code
GitHub Copilot reviewer fails	<code>github-copilot</code> user not a repo collaborator	Handled gracefully with try-catch; enable Copilot Code Review in repo Settings → Copilot

20.4 Async Config Thread Pools (cb-core AsyncConfig)

Executor Bean	Core Threads	Max Threads	Used By
<code>cbAgentExecutor</code>	2	4	FixGenerationService
<code>cbPatcherExecutor</code>	2	4	PatcherService
<code>cbPrExecutor</code>	2	4	PRService
<code>cbNotifierExecutor</code>	2	4	NotifierService